# TwinRouterBench: Fast Static and Live Dynamic Evaluation for Realistic Agentic LLM Routing

Pei Yang[1,*] Wanyi Chen[2,*] Tongyun Yang[3] Pengbin Feng[4] Jiarong Xing[5] Wentao Guo[1]
Yuhang Yao[6] Yuhang Han[7] Hanchen Li[8] Xu Wang[9] Zeyu Wang[10] Jie Xiao[1] Anjie Yang[1]
Liang Tian[1] Lynn Ai[1] Eric Yang[1] Tianyu Shi[1,†]

[1]Gradient [2]Soochow University [3]Independent Researcher [4]University of Southern California [5]Rice University
[6]Carnegie Mellon University [7]Shanghai Jiao Tong University
[8]University of California, Berkeley [9]University of the Chinese Academy of Sciences [10]University of California, Los Angeles
[*]Equal contribution [†]Corresponding author

LLM routing matters most in long-horizon applications such as coding agents, deep research systems, and computer-use agents, where a single user request triggers many model calls. Routing each call to the cheapest sufficient model can cut costs without sacrificing quality, yet existing router benchmarks evaluate routers only on one-shot prompts. They never expose the router-visible prefix at an intermediate agent step, never test whether a cheaper replacement preserves downstream task success, and often rely on online LLM judges at evaluation time. We introduce **TwinRouterBench**, a step-level routing benchmark with two tracks. The *static track* provides 970 router-visible prefixes from 520 instances across SWE-bench, BFCL, mtRAG, QMSum, and PinchBench, each paired with an execution-verified target tier estimated under a released downgrade-and-cascade protocol; scoring is deterministic arithmetic over tier labels, trajectory membership, and token costs, with no online evaluator-side LLM judge. The *dynamic track* supplies a harness that runs routers on the full 500-case SWE-bench Verified suite; in this paper we report a 100-case held-out evaluation disjoint from the static SWE supervision split. At each LLM call the router selects a concrete model from a locked pool, and success is measured by official task resolution and realized API spend. The two tracks support fast offline iteration followed by end-to-end validation under live agent execution. Code and data are available at https://github.com/CommonstackAI/TwinRouterBench.

**Date:** May 8, 2026
**Type:** arXiv preprint
**Correspondence:** tianyu@gradient.network
**Code and Data:** https://github.com/CommonstackAI/TwinRouterBench

## 1 Introduction

LLM routers select which model should handle each call [Feng et al., 2024, Stripelis et al., 2024]. In production multi-turn agents, each call carries a rich prefix—chat history, retrieved passages, shell logs, tool outputs, and partial code edits—and a cheaper choice at step $i$ may only fail downstream. Existing benchmarks evaluate routing on isolated one-shot prompts; they do not expose these prefixes or test whether a per-step decision breaks a later step.

The expensive unit in modern applications is rarely a one-shot query. Coding agents solve GitHub issues through many rounds of exploration, patching, and verification [Jimenez et al., 2024]; tool systems maintain state over serial and parallel calls [Patil et al., 2025]; retrieval and summarization workloads carry long multi-turn context [Katsis et al., 2025, Zhong et al., 2021]. A cheap model may suffice for a file lookup, while a later patch-writing step may require the strongest tier. A useful step-level benchmark must cover these

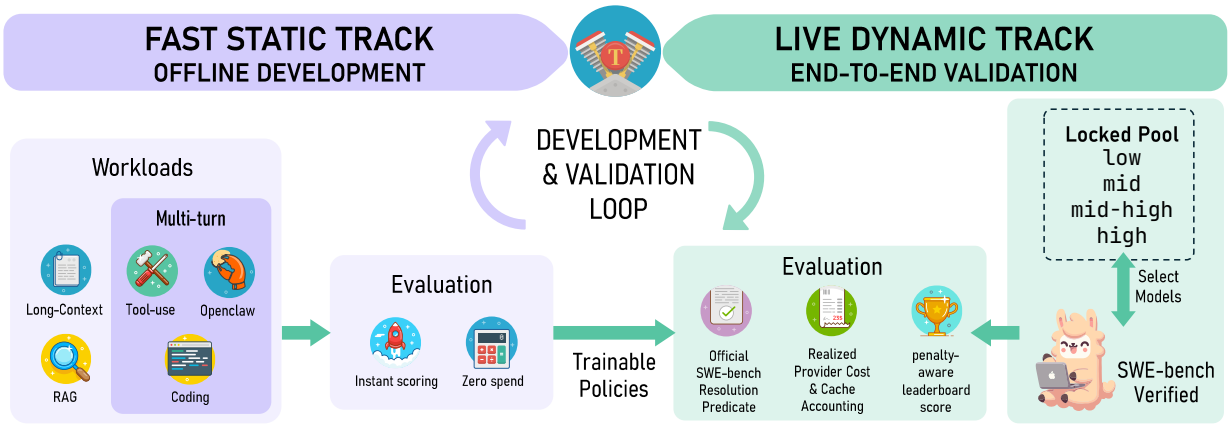


**Figure 1.** Overview of TwinRouterBench. The benchmark provides a fast static track for offline router development and a live dynamic track for end-to-end validation. The static track covers 970 step-level rows from 520 instances across five workloads, each with an execution-verified target tier; the dynamic track runs routers on SWE-bench Verified with realized API cost.

workloads, ground labels in execution outcomes rather than model-based judgement alone, and support both cheap deterministic offline scoring and end-to-end execution with realized API cost as the definitive validation. We therefore maintain two distinct tracks.

RouterBench, LLMRouterBench, and RouterArena evaluate query-level routing on complete one-shot prompts [Hu et al., 2024, Li et al., 2026, Lu et al., 2025]. Such a router can still be applied mid-trajectory by feeding the full conversation history as a single query, but its training and evaluation data are one-shot prompts, so neither tells us whether a cheap choice at step $i$ breaks a later step. We expose the full prefix a router observes before each LLM call (chat history, retrieved passages, tool outputs, shell traces, and code diffs) as a first-class input, and we verify each label by running the task to completion: if the trajectory still passes with the same number of steps after downgrading, we infer the replaced step was handled correctly. TRIM studies reasoning-step escalation in math [Kapoor et al., 2026]; our setting is agentic and multi-turn.

Building such a benchmark faces two main challenges. First, using a frontier model as the router is unreliable: in our Claude Opus 4.6 diagnostic, a single-template LLM router predicts `high` for only 7 of 147 verified-`high` SWE-bench steps and fails every SWE trajectory. Second, multi-turn trajectories create combinatorial label complexity: with $n$ candidate models and $m$ steps, exhaustive verification requires $n^m$ runs even when the trajectory length is fixed, because the prefix at step $i{+}1$ depends on the model chosen at step $i$. Practical label construction must therefore approximate this space with affordable protocols.

We introduce **TwinRouterBench**, a step-level routing benchmark with a fast static track for offline development and a live dynamic track for end-to-end validation.

**Contributions.**

- **A fast static track for realistic routing supervision.** The static track covers the workloads where step-level routing pays off most (long-context, multi-turn, tool-use, RAG, summarization, and code-repair) and exposes the exact prefix a router observes before each LLM call. Its 970 labels are execution-verified estimates under a fixed downgrade-and-cascade protocol with tier-pool cascade verification, mixed-prefix reconstruction, and manual audit on multi-step executable workloads. Scoring is deterministic arithmetic over labels, trajectory membership, and token costs: evaluation takes milliseconds and requires no online evaluator-side LLM judge. The same data also supports training; a logistic router trained

on the static labels achieves the best cost–success trade-off among the policies we evaluate.

- **A live dynamic track for software-agent routing.** The dynamic harness supports the full 500-case SWE-bench Verified suite; this paper reports a 100-case held-out evaluation disjoint from the static SWE supervision split. At each LLM request the router selects a concrete model from a locked pool; scoring uses the official SWE-bench resolution predicate, realized provider spend, cache accounting, and a flat unresolved-instance penalty in the leaderboard bill (§5.2), with no evaluator-side LLM judge.
- **A two-track development and validation loop.** The static track provides cheap, trainable supervision for rapid iteration. The dynamic track tests whether the resulting policy survives live execution. We report them side by side so that offline and end-to-end evidence remain distinguishable.

## 2 Related Work

**Query-level routing and cascades.** An early line of work treats model selection as an inference-time decision over quality and cost for a whole query. FrugalGPT learns a cascade over heterogeneous APIs to reduce cost [Chen et al., 2024]. AutoMix queries a smaller model first, estimates reliability via self-verification, and escalates when needed [Aggarwal et al., 2024]. Hybrid LLM routes between a local and a cloud model based on predicted difficulty [Ding et al., 2024]. RouteLLM trains routers from preference data and studies cross-pair transfer [Ong et al., 2025]. A recent survey formalizes the shared objective as selecting a model subject to a cost budget [Varangot-Reille et al., 2025]. These methods effectively abstract model selection on complete queries, but they leave the intermediate calls inside multi-turn agent trajectories under-explored, even though tool outputs and partial state already shape the prefix.

**Router benchmarks.** RouterBench collects over 405k inference outcomes across diverse single-prompt tasks, providing the first large-scale testbed for comparing routing algorithms [Hu et al., 2024]. LLMRouterBench scales to 400k+ instances, 21 datasets, and 33 models under unified performance and cost metrics [Li et al., 2026]; although it includes 500 SWE-Bench issues, SWE-Bench is configured as a single-query 0-shot PASS@1 task with no agent scaffold and no intermediate routable call. RouterArena adds an open comparison platform with difficulty levels and automated leaderboards [Lu et al., 2025]. Triage makes one tier decision per SWE-bench Lite issue using code-health features [Madeyski, 2026]. These benchmarks are valuable for comparing query-level routers, but even when multi-step tasks such as SWE-Bench are included, routing reduces to picking one model per issue from a pre-computed outcome matrix; the intermediate retrieval, analysis, and debugging calls within an issue cannot be routed individually to the cheapest sufficient model at each step.

**Step-level routing.** TRIM studies stepwise routing for multi-step math reasoning by identifying critical reasoning steps and escalating them to larger models, and shows that selective per-step upgrading can preserve accuracy while reducing average cost [Kapoor et al., 2026]. Math reasoning, however, is a relatively self-contained setting that lacks tool outputs, retrieval contexts, shell traces, and code state, all of which are central to real agent deployments. TRIM also derives labels from hypothetical reasoning traces without executing the downgraded trajectory end-to-end to verify task success. For executable long-context agent workloads the core labeling problem persists: given the current prefix, which model tier is cheap enough yet still preserves final task resolution?

## 3 Problem Formulation and Benchmark Overview

### 3.1 Step-level routing

A multi-turn LLM agent produces a trajectory $\tau = (s_1, s_2, \ldots, s_N)$ of LLM calls. At step $i$, the agent holds a message history $M_i = [m_1, \ldots, m_k]$ (a *prefix*: system prompt, user instruction, previous assistant messages, tool outputs, retrieval chunks), emits a new assistant message $a_i = \mathcal{R}(M_i)$, executes any tool calls in $a_i$ against an external environment to produce observation $o_i$, and appends $a_i, o_i$ to the history: $M_{i+1} = M_i \parallel [a_i, o_i]$. The agent halts when it submits or when a budget is exhausted.

### 3.2 Conditional per-call routing

A standard LLM router maps a query to a candidate model to optimize quality and cost; recent surveys formalize this as selecting from a model set subject to a cost budget [Varangot-Reille et al., 2025]. TwinRouterBench studies the conditional per-call version: the query is the full router-visible prefix $x_i$ before the next LLM call, including system messages, user requests, prior assistant messages, tool outputs, retrieval snippets, logs, and partial code edits. A step-level router is a conditional decision rule

$$\pi : x_i \mapsto t_i \in \mathcal{T},$$

where

$$\mathcal{T} = \{\texttt{low}, \texttt{mid}, \texttt{mid\_high}, \texttt{high}\}$$

is an ordered set of cost and capability tiers. Each tier maps to a pool of models with similar capability and cost; the concrete model is decided downstream by a tier-to-model mapping, keeping public rows free of vendor model IDs.

The ideal conditional target tier is the cheapest tier that can correctly handle the current step given the prefix $x_i$. Let $\mathcal{M}_t \subseteq \mathcal{M}$ denote the model pool for tier $t$, and let $V_i(m; x_i) = 1$ indicate that model $m$ produces a sufficient response for the current step under prefix $x_i$. For one-shot workloads, $V_i$ is the task-level success predicate. For multi-turn agentic workloads, $V_i(m; x_i) = 1$ means that model $m$ correctly solves the problem posed at step $i$; when each step is solved correctly, the trajectory as a whole is more likely to succeed. The ideal target is

$$t_i^\star = \min \{t \in \mathcal{T} : \exists m \in \mathcal{M}_t \text{ such that } V_i(m; x_i) = 1\}.$$

$t_i^\star$ is a conditional per-call quantity, not a globally optimal joint policy over the full trajectory. At deployment time a router observes only the current prefix $x_i$: it cannot change earlier outputs and cannot choose future calls before their prefixes exist.

Because per-step correctness cannot always be judged in isolation, we approximate $V_i$ through end-to-end execution: a downgraded step is accepted when the full trajectory still passes with the same number of steps, which lets us infer that the replaced step was handled correctly. The released label $\hat{t}_i$ is therefore an execution-verified estimate of $t_i^\star$ under our fixed downgrade-and-cascade protocol (§4), supplemented by a manual audit that cross-checks this inference on a random sample of steps (§4, Manual audit). A tier $t$ is accepted when at least one model in $\mathcal{M}_t$ resolves the mixed-model trajectory. We use a fixed 11-model pool spanning four tiers; the full list and grouping are in the appendix (Table A4). Changing the pool, tier membership, harness, or verification protocol changes $\hat{t}_i$ and constitutes a new benchmark version.

### 3.3 Corpus

The release contains 970 step-level rows from 520 trajectory instances across five benchmarks; per-benchmark instance and step counts (and prefix-token medians) are summarized in Appendix Table A2. A

trajectory is one successful end-to-end run of a strong model on the original task; each step corresponds to one LLM call inside that trajectory, and its prefix is exactly the messages the router would see before that call (§4 describes the full construction). Every row has the shape `id`, `benchmark`, `instance_id`, `step_index`, `total_steps`, `messages`, `target_tier`, and `target_tier_id`. No vendor model IDs appear in the public JSONL, only the tier label. Full row examples are in the appendix.

Appendix Table A3 summarizes the distribution of cost-saving opportunities across tiers. The `low` tier tests whether a router can avoid unnecessary expensive calls while preserving task success. The two middle tiers are transitional capability bands; routers with three output classes can merge them into a single middle band or map through the released tier ids. SWE-bench contributes most `high`-tier rows, signaling when a router must stay conservative in difficult code-repair steps.

mtRAG and QMSum contribute single-call rows whose prefixes may contain multi-turn context. BFCL contains both single-turn and multi-turn function calling: 130 instances yield 248 rows, with 29 instances having more than one routed step. SWE-bench and PinchBench contribute multi-step agent traces. We include all cases because production routers encounter both single-call long-context states and sequential agent states.

### 3.4 Static evaluation without an online judge

Static scoring is deterministic arithmetic over tier labels, trajectory membership, and token costs; the four metrics RowPass, RowExact, TrajPass, and CostSave are defined in §5.1.

### 3.5 Dynamic SWE-bench track

The dynamic track provides a harness that supports the full 500-case SWE-bench Verified suite; this paper reports a 100-case held-out evaluation disjoint from any static-track SWE-bench case. At each LLM call the router chooses a concrete model from the locked pool given the current prefix, available models, cache state, and budget. The track reports three results: resolve rate (the official SWE-bench predicate), realized API spend, and a leaderboard bill that adds the same flat unresolved-instance penalty on top of API cost for every policy (§5.2).

## 4 Dataset Construction

Figure 2 illustrates how each multi-turn case is progressively downgraded from a successful strong-model trajectory to produce per-step verified tier labels through execution-verified search.

**Seed from successful strong trajectories.** We first collect trajectories that resolve under a strong-model setting. For SWE-bench, Claude Opus 4.6 or GPT-5.4 running through `mini-swe-agent` must submit a patch that passes the official `FAIL_TO_PASS` tests; BFCL, mtRAG, QMSum, and PinchBench [Kilo Code, 2026] use their respective harness pass predicates. Failed seeds are dropped so that routing labels do not conflate task difficulty with routing error.

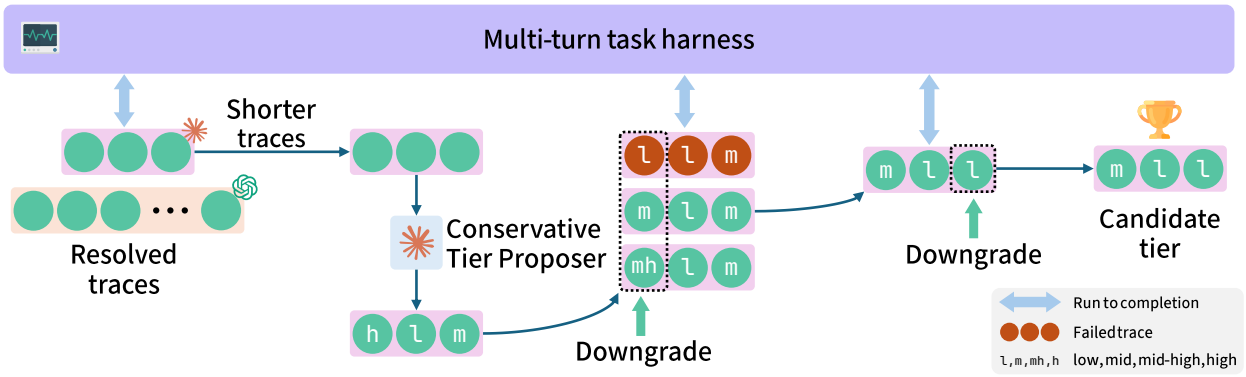


**Figure 2.** TwinRouterBench construction pipeline. For each multi-turn case, the pipeline starts from a successful strong-model trajectory and progressively downgrades individual steps to cheaper tiers via execution-verified search, producing the verified tier label for every LLM call in the trace.

**Search lower tiers under causal prefixes.** For each surviving trajectory, Claude Opus 4.6 provides a search hint: whether a step appears downgradeable and, if so, the most aggressive lower tier $h_i$ to probe first. The hint is a pruning device, never a label. We then run a greedy sequential-locking downgrade search (Appendix Algorithm A1). At step $i$, previously locked steps keep their verified tiers, future steps use the strong tier, and candidates for step $i$ are tried from cheapest to strongest starting at $h_i$. Because per-step correctness cannot always be observed directly, we accept a candidate when the full trajectory still passes with the same number of steps, inferring that the downgraded step was handled correctly; if no candidate passes, the step stays at the strong tier. This reduces the naive $|T|^N$ grid to $\mathcal{O}(|T|N)$ trials while ensuring later prefixes contain actual degraded outputs from earlier routed steps. Locked prefixes are replayed from a response cache keyed by instance, step, model, and generation parameters; cache entries are invalidated when an upstream lock changes.

**Verify tiers as pools, not single models.** A tier label should mean that the tier pool can solve the step, not that one model happened to pass. We run a three-model cascade for non-`low` labels: a tier is accepted if any of the three pool models passes under the mixed-model trace. In a SWE-bench last-step audit, the cascade recovered 28 of 33 downgradeable labels versus 15 of 33 under single-model probing, confirming that single-representative search is too brittle for supervision.

**Harden open-ended pass predicates.** Some benchmarks (mtRAG, QMSum) rely on LLM-as-judge evaluation whose default prompts and rubrics are not strict enough. During early development we manually inspected a random sample of judge verdicts and found pseudo-passes where surface-level similarity masks incorrect or incomplete answers. We replace these weak reference-similarity rubrics with a structured judge that first checks whether the current turn is solved, then scores Faithfulness, Appropriateness, and Completeness, with evidence-conflict hard failures and a conservative fail-on-uncertainty default. We calibrate the tightened rubric against Radbench-style thresholded ratings [Kuo et al., 2025] and verify through iterative test runs that the hardened judge eliminates the pseudo-pass patterns discovered in our initial sample; structurally unreliable mtRAG samples and all-tier failures are discarded. SWE-bench and BFCL already have executable or structural pass predicates and need no additional hardening.

**Manual audit.** Our downgrade search fixes all other steps and tests one step at a time. This greedy strategy reduces computational cost from exponential to linear but may miss cross-step interaction effects: a combination of two downgrades might fail even though each succeeds individually. To guard against such cases, we conduct a human sampling audit over the three workloads with multi-step trajectories

| Benchmark | Total steps | Sampled steps | Sampling % | Downgradeable | Tight % |
|---|---|---|---|---|---|
| BFCL | 248 | 25 | 10.1 | 0 | 100.0 |
| SWE-bench | 336 | 34 | 10.1 | 1 | 97.1 |
| PinchBench | 48 | 5 | 10.4 | 0 | 100.0 |
| Total | 632 | 64 | 10.1 | 1 | 98.4 |

**Table 1.** Step-level manual audit summary. *Downgradeable*: the reviewer believes the verified tier can be lowered one more level without breaking task resolution. *Tight %*: the share already at the conditional optimum. We sample routed steps from the three workloads with multi-step trajectories; mtRAG and QMSum are single-turn tasks already guarded by the hardened judge and are not resampled.

(BFCL, SWE-bench, PinchBench), drawing an independent ∼10% random sample of routed steps from each. mtRAG and QMSum are single-turn tasks that lack multi-turn compounding complexity; their labels are already constrained by the hardened judge whose pseudo-pass patterns were eliminated through the iterative calibration described above, so they are not resampled in the final audit. For each sampled step a reviewer freezes the router-visible prefix and judges whether the tier can be lowered one more level, labelling it as *tight*, *uncertain*, or *further-downgradeable*. The audit covers 64 steps (25 BFCL, 34 SWE-bench, 5 PinchBench): 63 were judged tight; one SWE-bench step was flagged as further-downgradeable and its tier was lowered before release (Table 1). Details of the audit protocol are in Appendix A.13.

**Release criterion.** No row enters the static track unless (1) the strong seed trajectory passed, (2) the tier pool contains a model that passes under the mixed trace, and (3) for open-ended workloads (mtRAG, QMSum), the hardened judge does not flag the answer as a pseudo-pass. Each label is an execution-verified estimate of the cheapest tier that can correctly handle the current step: since direct per-step judgement is not always possible, we infer per-step sufficiency from end-to-end trajectory pass with a fixed step count, and cross-check through manual audit.

# 5 Evaluation Method, Baselines and Experiment Setup

## 5.1 Static scoring protocol

A per-request tier label alone does not capture the downstream cost of choosing the wrong tier at step $k$ of an $N$-step trajectory. We score each router on four metrics that combine a step-level view with a trajectory-level view: RowPass, RowExact, TrajPass, and CostSave. The first two operate per row. TrajPass requires every routed step in a trajectory to pass. CostSave measures savings relative to the always-`high` baseline but only credits savings on passing trajectories; API cost on failed trajectories is charged as burned. The detailed derivation is in the appendix.

Let $\hat{t}_i$ be the released target tier for row $i$ and $\tilde{t}_i$ the router prediction. A row passes if $\tilde{t}_i \geq \hat{t}_i$ and is exact if $\tilde{t}_i = \hat{t}_i$. A trajectory passes only if every routed row passes. Let $c_i(t)$ be the tier-accounting cost for row $i$ at tier $t$ and $T_3$ denote `high`. For workload $b$, the always-`high` bill is $D_b = \sum_{i \in b} c_i(T_3)$. The numerator $N_b$ accumulates at the trajectory level: passing trajectories contribute $\sum_i [c_i(T_3) - c_i(\tilde{t}_i)]$; failing trajectories contribute $-\sum_i c_i(\tilde{t}_i)$. The grader reports

$$\text{CostSave} = \sum_b w_b \frac{N_b}{D_b}, \qquad w_b = \frac{n_b}{970},$$

where $n_b$ is the number of rows in workload $b$, so workloads are row-weighted after each cost ratio is

computed. An always-`high` policy scores CostSave $= 0$ by construction, since $N_b = 0$ for every workload.

**Headline score.**

$$\text{Combined} = \tfrac{1}{4}(\text{RowPass} + \text{RowExact} + \text{TrajPass} + \text{CostSave}).$$

The four components are the primary reporting unit; Combined is a single operating point under equal weighting. We justify the failure-aware cost accounting with an ablation in Appendix A.15.

**Pricing and token accounting.** The `high` tier is 10–50× more expensive than the lower tiers (Appendix Table A5), so naive per-step cost accounting overrates conservative routers. These are static tier-level constants; dynamic-pool model prices are in Appendix A.7. Prompt tokens are split into `cache_read`, `cache_write`, and fresh `input`; output tokens are counted from the recorded transcript. Token counting uses native vendor tokenizers where available and `cl100k_base` as fallback. For row $i$ and tier $t$, the scorer uses the same four-bucket form as real API billing:

$$c_i(t) = \frac{n_i^{\text{in}} p_t^{\text{in}} + n_i^{\text{cr}} p_t^{\text{cr}} + n_i^{\text{cw}} p_t^{\text{cw}} + n_i^{\text{out}} p_t^{\text{out}}}{10^6},$$

where the token counts $n_i^{\cdot}$ and the per-tier rates $p_t^{\cdot}$ follow Appendix Table A5. The static grader enables prompt-cache modeling throughout: prompt tokens are billed as cache_write on cold starts, tier switches, TTL expiry (5 minutes, matching provider defaults such as Anthropic's prompt-cache policy), or prefix deltas, and as cache_read on valid same-tier prefix hits; the fresh-input term is retained to match the four-bucket dynamic accounting interface.

**Accounting for routing-specific effects.** Mid-trajectory model switching raises three common concerns: cache misses on tier changes, out-of-distribution behavior under mixed-model histories, and tool-format mismatch across model families. Our evaluation prices each in rather than assuming them benign. Cache writes on tier switch are charged at the *incoming* tier's rate (cache_write at `high` is 12.5× its cache_read in Appendix Table A5), so a policy that churns into expensive tiers incurs the cache_write cost at list price; the dynamic track then reports realized OpenRouter cost rather than simulated cost, and a router selecting a model per-step still reduces realized API cost by 53.1% relative to unrouted Opus despite those switches (Table 3). Out-of-distribution behavior in mixed-model trajectories is handled by execution-verified labels (§4) and the failure-aware numerator: unresolved trajectories are billed at API cost plus a flat penalty cost, so OOD failures cannot appear as savings. Tool-format heterogeneity (e.g. structured `apply_patch` calls versus interleaved `str_replace_editor` edits) is contained by the shared `mini-swe-agent` harness, which derives every final patch via `git diff` at termination; residual stylistic drift surfaces as trajectory failure under the same cost formula.

## 5.2 Dynamic SWE-bench scoring

The dynamic track is scored independently from the static proxy. A dynamic run executes SWE-bench Verified instances end-to-end using `mini-swe-agent` v2.2.8 as the agent harness; at each LLM call the router selects a concrete model id from the locked pool. Per-step cost uses the same four-bucket formula, but $p$ values are live per-model OpenRouter prices (Appendix A.7) and token buckets come from provider usage logs normalized to `input`, `cache_read`, `cache_write`, and `output`. Let $A_j = \sum_{i \in \tau_j} c_i(m_i)$ denote the realized API cost on instance $j$, where $m_i$ is the model selected at step $i$, and let $\text{resolved}_j \in \{0, 1\}$ follow the official SWE-bench predicate. The per-instance leaderboard bill is

$$\text{bill}_j = A_j + (1 - \text{resolved}_j)\,\gamma,$$

where $\gamma$ is a fixed USD add-on per unresolved instance (release default $\gamma$=0.60, set slightly above the observed per-instance average API cost of the unrouted Opus 4.6 baseline, $0.55). We model $\gamma$ as the per-case price of a hypothetical perfect solver that resolves any SWE-bench instance; this applies uniformly to every policy including the unrouted baseline, so the leaderboard bill reflects both realized API cost and the remediation cost of failures. Resolved instances incur only $A_j$. The leaderboard sort key (lower is better) is $\sum_j \text{bill}_j$ (`total_leaderboard_bill_usd`); `total_router_cost_usd` and `total_penalty_cost_usd` report the API cost and penalty cost portions separately. Instances excluded as infra failures (optional flag in the scorer) carry no add-on in headline totals. Static and dynamic scores are never averaged: the static track covers five workloads while the dynamic track covers SWE-bench Verified only.

### 5.3 Baselines

We report initial static reference points: SR-KNN [Liu et al., 2026], an in-sample 1-nearest-neighbor upper bound over question-bank embeddings; ClawRouter [BlockRunAI, 2026], an open rule-based router with LLM fallback disabled; and UncommonRoute [Commonstack AI, 2026], a public rule-based router whose upstream defaults we adopt as the rule-based baseline; the trained variant below is trained on top of its interface. Predictions are produced offline and scored by the public grader. We also run Claude Opus 4.6 as a separate LLM-as-router diagnostic with `max_tokens`= 1 and prompt caching; malformed non-digit responses count as errors.

For the trained UncommonRoute variant, we train only the routing policy, not the underlying language models. The latest user request is encoded with a frozen `BAAI/bge-small-en-v1.5` sentence embedding and concatenated with routing-time metadata (message count, tool-call presence, tool-message count, request length, code/question indicators). A multinomial logistic regression classifier with L2 regularization is trained on the training split, with regularization strength selected by 5-fold cross-validation; the embedding model is not fine-tuned. A separate calibration split fits confidence parameters, and performance is reported on a held-out split not used for training or calibration (Table 3). The appendix gives the rule-based configuration; the release package contains the locked data and scoring code to reproduce all tables.

## 6 Benchmark Results & Analysis

### 6.1 Static-Track Diagnostic Results

Table 2 reports static-track scores on all 970 rows. SR-KNN [Liu et al., 2026] is an in-sample upper-bound reference and ranks first on COMBINED among the three data-driven routers above the separator (the Opus 4.6 (always-`high`) row is a tier-`high` cost-envelope reference, not a held-out supervised baseline). Dynamic-track results are reported separately in §6.2.

ClawRouter (rule-based) has only 11.75% ROWEXACT but 53.82 COSTSAVE: it routes one tier above the verified label on most non-`high` rows, which destroys exact match but stays far cheaper than always-`high`. The cost is trajectory failure on SWE-bench (38/40 failed). UncommonRoute (rule-based) has much higher ROWEXACT yet only 15.99 COSTSAVE because it jumps to `high` 217 times on non-`high` rows and still fails 39/40 SWE trajectories, pulling the SWE slice to −86.08 COSTSAVE. SR-KNN achieves similar COSTSAVE as ClawRouter (rule-based) but with much stronger trajectory preservation. The failure-aware COSTSAVE formula (§5.1) ensures that cheap calls on failed trajectories are charged rather than credited; the ablation in Appendix A.15 confirms that looser formulas would incorrectly reward ClawRouter (rule-based) for its one-tier upgrades.

| Router | RowPass ↑ | RowExact ↑ | TrajPass ↑ | CostSave ↑ | **Combined ↑** |
|---|---|---|---|---|---|
| SR-KNN (in-sample ub.) | **91.86** | **78.76** | **84.74** | **56.18** | **77.89** |
| ClawRouter (rule-based) | 80.82 | 11.75 | 64.64 | 53.82 | 52.76 |
| UncommonRoute (rule-based) | 88.35 | 61.13 | 62.37 | 15.99 | 56.96 |
| Opus 4.6 (always-`high`) | 100.00 | 17.53 | 100.00 | 0.00 | 54.38 |

**Table 2.** Static-track diagnostic results (970 rows, all percentages). SR-KNN is an in-sample upper bound. The Opus 4.6 (always-`high`) row is the tier-`high` envelope used by the static CostSave denominator: RowPass and TrajPass are 100% because $\tilde{t} = \texttt{high} \geq \hat{t}$ on every row; RowExact is 170/970 (only the 170 verified-`high` rows match); CostSave is 0 by construction (§5.1). Opus 4.6 as an LLM-as-router is analyzed separately in §6.1 because that run predates the failure-aware scoring formulation.

| Policy | Resolved ↑ | Avg. API cost ↓ | API cost ↓ | Penalty cost ↓ | Leaderboard bill ↓ |
|---|---|---|---|---|---|
| SR-KNN router | **75**/100 | $0.56 | $55.61 | $15.00 | $70.61 |
| UncommonRoute (trained) | **75**/100 | **$0.26** | **$25.66** | $15.00 | **$40.66** |
| UncommonRoute (rule-based) | 73/100 | $1.73 | $172.56 | $16.20 | $188.76 |
| Opus 4.6 (no routing) | 74/100 | $0.55 | $54.73 | $15.60 | $70.33 |

**Table 3.** Dynamic-track results on a 100-case held-out SWE-bench Verified split. API cost is the realized routed API spend; Penalty cost is $0.60× the number of unresolved instances, where $\gamma$=0.60 is the per-case price of a hypothetical perfect solver (§5.2); Leaderboard bill (sort key, lower is better) is API cost + Penalty cost. The penalty applies uniformly to every policy. The unrouted Opus 4.6 baseline is separated below the routing policies, mirroring the cost-envelope row in Table 2.

A per-workload breakdown of static-track results is in Appendix A.14.

**Opus-as-router case study.** We ran Claude Opus 4.6 over the full benchmark with a prompt describing the 11-model pool and their SWE-bench resolve rates. On the 300 valid SWE-bench rows, Opus predicts `high` for only 7 of 147 verified-`high` steps and fails all 40 SWE trajectories. This is a case study of one frontier model under one prompt template, not a general claim about all LLM routers, but it illustrates that frontier-model prompting does not substitute for execution-verified step-level supervision. The full prediction breakdown is in Appendix A.11 (Table A9).

## 6.2 Dynamic Held-out SWE Execution

Because each dynamic run executes a full SWE-bench agent end-to-end with live API calls, cost scales linearly with the number of instances. To keep evaluation affordable while still testing generalization, we randomly sample 100 SWE-bench Verified instances that do not overlap with any static-track SWE-bench case and run all policies on this shared held-out set. Table 3 reports the results. Routing supervision differs across policies: Opus and UncommonRoute (rule-based) use no offline routing labels; SR-KNN uses the upstream `semantic-router` defaults [Liu et al., 2026]; only UncommonRoute (trained) is fit on static-track labels (§5.3).

At comparable resolve rate, trained UncommonRoute reduces API cost by $1 - 25.66/54.73 = 53.1\%$ relative to unrouted Opus, and costs 6.7× less than the rule-based variant (Table 3). Because the static corpus is small, we validate the trained router through end-to-end dynamic execution rather than reporting static held-out scores. This provides initial evidence that the static labels can train a router that substantially reduces cost while maintaining SWE-bench resolution on this 100-case held-out split.

# 7 Conclusion and Limitations

We release TwinRouterBench, a step-level routing benchmark built on the scenarios where routing matters most: long-context, multi-turn agent workloads such as SWE-bench, PinchBench, BFCL, mtRAG, and QMSum. The static track approximates per-step cheapest-sufficient tiers via a greedy sequential-locking downgrade search, and a manual audit on multi-turn trajectories confirms the reliability of the resulting labels. On the dynamic track, a 100-case held-out SWE-bench evaluation demonstrates that the live execution framework produces meaningful end-to-end results. A logistic router trained on the static labels achieves a 53% cost reduction at matched resolve rate on the dynamic track, showing that the static data serves not only as a fast offline evaluation tool but also as effective training supervision for practical routers.

**Limitations.** The static corpus of 970 routed steps from 520 instances is sufficient for initial diagnostics and training but not exhaustive across agent workloads, domains, or routing patterns. Labels are estimated under the fixed model pool, cascade protocol, and price frontier of §5.1, and would need updating if a future model became both cheap and highly capable. Dynamic coverage beyond SWE-bench Verified, and keeping data current as pools and pricing evolve, are future work.

**Broader impacts.** TwinRouterBench may accelerate the development of cost-effective LLM routers by providing standardized step-level evaluation, indirectly reducing API cost and energy consumption in agentic deployments. The two-track design also improves reproducibility by separating deterministic offline scoring from live validation. A potential negative impact is benchmark overfitting: community efforts may concentrate on the current 970-row corpus and five workloads at the expense of uncovered domains, languages, or agent architectures. Additionally, tier labels are tied to a fixed model pool and price snapshot; as models and pricing evolve, stale labels could mislead router training or evaluation. We mitigate these risks through versioned model pools and pricing, explicit scope limitations, and a release design that supports pool updates and re-labeling.

## References


Tao Feng, Yanzhen Shen, and Jiaxuan You. GraphRouter: A graph-based router for LLM selections, 2024. URL https://arxiv.org/abs/2410.03834.

Dimitris Stripelis, Zhaozhuo Xu, Zijian Hu, Alay Dilipbhai Shah, Han Jin, Yuhang Yao, Jipeng Zhang, Tong Zhang, Salman Avestimehr, and Chaoyang He. TensorOpera router: A multi-model router for efficient LLM inference. In Franck Dernoncourt, Daniel Preoţiuc-Pietro, and Anastasia Shimorina, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 452–462, Miami, Florida, US, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-industry.34. URL https://aclanthology.org/2024.emnlp-industry.34/.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): from tool use to agentic evaluation of large language models. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR / OpenReview.net, 2025. URL https://proceedings.mlr.press/v267/patil25a.html.

Yannis Katsis, Sara Rosenthal, Kshitij Fadnis, Chulaka Gunasekara, Young-Suk Lee, Lucian Popa, Vraj Shah, Huaiyu Zhu, Danish Contractor, and Marina Danilevsky. mtrag: A multi-turn conversational benchmark for evaluating retrieval-augmented generation systems. *Trans. Assoc. Comput. Linguistics*, 13:784–808, 2025. doi: 10.1162/TACL.A.19. URL https://doi.org/10.1162/tacl.a.19.

Ming Zhong, Da Yin, Tao Yu, Ahmad Zaidi, Mutethia Mutuma, Rahul Jha, Ahmed Hassan Awadallah, Asli Celikyilmaz, Yang Liu, Xipeng Qiu, and Dragomir R. Radev. QMSum: A new benchmark for query-based multi-domain meeting summarization. In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tür, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou, editors, *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021*, pages 5905–5921. Association for Computational Linguistics, 2021. doi: 10.18653/V1/2021.NAACL-MAIN.472. URL https://doi.org/10.18653/v1/2021.naacl-main.472.

Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. RouterBench: A benchmark for multi-LLM routing system. *CoRR*, abs/2403.12031, 2024. doi: 10.48550/ARXIV.2403.12031. URL https://doi.org/10.48550/arXiv.2403.12031.

Hao Li, Yiqun Zhang, Zhaoyan Guo, Chenxu Wang, Shengji Tang, Qiaosheng Zhang, Yang Chen, Biqing Qi, Peng Ye, Lei Bai, et al. LLMRouterBench: A massive benchmark and unified framework for LLM routing. arXiv preprint arXiv:2601.07206, 2026. URL https://arxiv.org/abs/2601.07206.

Yifan Lu, Rixin Liu, Jiayi Yuan, Xingqi Cui, Shenrun Zhang, Hongyi Liu, and Jiarong Xing. RouterArena: An open platform for comprehensive comparison of LLM routers. *CoRR*, abs/2510.00202, 2025. doi: 10.48550/ARXIV.2510.00202. URL https://doi.org/10.48550/arXiv.2510.00202.

Vansh Kapoor, Aman Gupta, Hao Chen, Anurag Beniwal, Jing Huang, and Aviral Kumar. TRIM: hybrid inference via targeted stepwise routing in multi-step reasoning tasks. *CoRR*, abs/2601.10245, 2026. doi: 10.48550/ARXIV.2601.10245. URL https://doi.org/10.48550/arXiv.2601.10245.

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*, 2024. URL https://openreview.net/forum?id=cSimKw5p6R.

Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, Shyam Upadhyay, Manaal Faruqui, and Mausam. AutoMix: Automatically mixing language models. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 131000–131034. Curran Associates, Inc., 2024. doi: 10.52202/079017-4164. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/ecda225cb187b40ea8edc1f46b03ffda-Paper-Conference.pdf.

Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM: cost-efficient and quality-aware query routing. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=02f3mUtqnM.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs from preference data. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL https://openreview.net/forum?id=8sSqNntaMr.

Clovis Varangot-Reille, Christophe Bouvard, Antoine Gourru, Mathieu Ciancone, Marion Schaeffer, and François Jacquenet. Doing more with less: A survey on routing strategies for resource optimisation in large language model-based systems. *CoRR*, abs/2502.00409, 2025. doi: 10.48550/ARXIV.2502.00409. URL https://doi.org/10.48550/arXiv.2502.00409.

Lech Madeyski. Triage: Routing software engineering tasks to cost-effective LLM tiers via code quality signals. arXiv preprint arXiv:2604.07494, 2026. URL https://arxiv.org/abs/2604.07494.

Kilo Code. PinchBench: Real-world benchmarks for AI coding agents. https://github.com/pinchbench/skill, 2026. Version 2.0.0. MIT license.

Tzu-Lin Kuo, FengTing Liao, Mu-Wei Hsieh, Fu-Chieh Chang, Po-Chun Hsu, and Da-shan Shiu. RAD-bench: Evaluating large language models' capabilities in retrieval augmented dialogues. In Weizhu Chen, Yi Yang, Mohammad Kachuee, and Xue-Yong Fu, editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track)*, pages 868–902, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-194-0. doi: 10.18653/v1/2025.naacl-industry.66. URL https://aclanthology.org/2025.naacl-industry.66/.

Xunzhuo Liu, Huamin Chen, Samzong Lu, Yossi Ovadia, Guohong Wen, Hao Wu, Zhengda Tan, Jintao Zhang, Senan Zedan, Yehudit Kerido, Liav Weiss, Haichen Zhang, Bishen Yu, Asaad Balum, Noa Limoy, Abdallah Samara, Baofa Fan, Brent Salisbury, Ryan Cook, Zhijie Wang, Qiping Pan, Rehan Khan, Avishek Goswami, Houston H. Zhang, Shuyi Wang, Ziang Tang, Fang Han, Zohaib Hassan, Jianqiao Zheng, and Avinash Changrani. vLLM semantic router: Signal driven decision routing for mixture-of-modality models, 2026. URL https://arxiv.org/abs/2603.04444. Version 0.3.0. Code: https://github.com/vllm-project/semantic-router.

BlockRunAI. ClawRouter. https://github.com/BlockRunAI/ClawRouter, 2026. Version 0.12.149. Accessed: 2026-04-28.

Commonstack AI. UncommonRoute: An open-source LLM routing library. https://github.com/CommonstackAI/UncommonRoute, 2026. Version 0.7.15. Accessed: 2026-04-28.

Emily M. Bender and Batya Friedman. Data statements for natural language processing: Toward mitigating system bias and enabling better science. *Transactions of the Association for Computational Linguistics*, 6:587–604, 2018. doi: 10.1162/tacl_a_00041. URL https://aclanthology.org/Q18-1041/.

Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723.

# A Appendix

## A.1 Dataset Card

**Table A1.** Dataset card for the TwinRouterBench static track. This follows the Bender and Friedman [2018] and Gebru et al. [2021] data statement conventions adapted for LLM benchmark releases.

| Field | Value |
|---|---|
| **Dataset name** | TwinRouterBench v1.0 (static track) |
| **Task type** | Step-level LLM routing (4-class classification over tier IDs 0–3) |
| **Size** | 970 rows from 520 trajectory instances |
| **Source workloads** | SWE-bench (Apache-2.0), BFCL (CC BY 4.0), mtRAG (IBM Research; see original license), QMSum (MIT), PinchBench [Kilo Code, 2026] (MIT; the 48 derived rows from 12 of its 53 tasks are included in this release and covered by the package-level Apache-2.0 license below) |
| **Provenance** | Message prefixes extracted from successful agent trajectories under strong-model execution; tier labels produced by greedy sequential-locking downgrade search and execution verification |
| **Label semantics** | Per-step cheapest-sufficient-tier estimate: the lowest tier whose pool contains a model that correctly handles the current step, verified by end-to-end trajectory pass with fixed step count under the 11-model pool, Opus-initialized sequential-locking downgrade search, and cascade protocol of §4, cross-checked by manual audit |
| **License** | Apache-2.0 |
| **Intended uses** | Training and evaluating step-level LLM routers; benchmarking agent deployment cost reduction |
| **Out-of-scope uses** | Claims of absolute routing optimality; deployment to model pools substantially different from the released tier map without re-labeling; use as a general SWE-bench or BFCL capability benchmark |
| **Potential biases** | Over-represents agentic code-repair (SWE-bench) at the `high` tier; BFCL/mtRAG/QMSum are nearly saturated at `low`; English-only content; labels are pool- and harness-specific |
| **Maintenance plan** | New tier-pool manifests and score protocol versions will be tracked via GitHub releases; tier map and pricing are versioned; current release is v1.0 |
| **Human oversight** | Final human audit of a ∼10% random step sample from BFCL, SWE-bench, and PinchBench (64 steps total); one SWE-bench step was flagged as further-downgradeable and its verified tier was lowered by one tier before release; no other still-downgradeable label was found in the audited sample |
| **Release URL** | Public code and data package: `https://github.com/CommonstackAI/TwinRouterBench` |

License metadata follows upstream documentation as of the release date; users should verify upstream terms before redistributing derived artifacts.

## A.2 Corpus overview

## A.3 Target-tier distribution and sequential-locking search

For space, we defer the workload target-tier counts (referenced from §3) and the sequential-locking downgrade search pseudocode (referenced from §4) to this appendix.

| Benchmark | Instances | Steps | Steps/trace (median) | Prefix tokens (median) | Scenario |
|---|---|---|---|---|---|
| SWE-bench | 40 | 336 | 9.0 | ∼5,300 | GitHub issue code repair (multi-step agent) |
| BFCL | 130 | 248 | 1.0 | ∼1,600 | Function calling (single- + multi-turn) |
| mtRAG | 193 | 193 | 1 | ∼1,900 | Multi-turn retrieval-augmented QA |
| QMSum | 145 | 145 | 1 | ∼3,000 | Query-focused meeting summarization |
| PinchBench | 12 | 48 | 4 | ∼10,500 | 23-task general agent suite |
| **Total** | **520** | **970** | | | |

**Table A2.** Corpus overview. Prefix-token medians are measured on the router-visible `messages` field using the Anthropic tokenizer.

| Benchmark | `low` | `mid` | `mid_high` | `high` |
|---|---|---|---|---|
| BFCL | 239 | 8 | 1 | 0 |
| mtRAG | 183 | 8 | 1 | 1 |
| QMSum | 132 | 10 | 3 | 0 |
| PinchBench | 41 | 3 | 3 | 1 |
| SWE-bench | 94 | 33 | 41 | 168 |
| Total | 689 | 62 | 49 | 170 |

**Table A3.** Target-tier distribution by workload.

## A.4 Artifact Structure

The reviewer package contains the static JSONL, locked manifests, scoring code, dynamic split, and compact dynamic summaries needed to inspect the benchmark and recompute the reported tables. The expected layout is:

```
data/static/question_bank.jsonl
data/static/manifest.json
data/dynamic/model_pool.json
data/dynamic/model_pricing.json
data/dynamic/tier_to_model.json
data/dynamic/ttl_policy.json
main/eval/
main/metrics.py
README.md
```

The static file contains 970 rows and exposes only tier labels, not vendor model IDs in individual records. Dynamic scoring uses the locked pool, pricing, tier map, and TTL policy files under `data/dynamic/`; held-out SWE-bench IDs and aggregate outputs are provided with the release artifacts. The README gives installation and smoke-test commands for the static grader and dynamic scoring CLI. For E&D-track submission, the release package also includes `metadata/croissant.json` with Croissant core and Responsible AI fields, license and source-workload metadata, and executable smoke-test scripts for the static grader and dynamic-summary scorer.

## A.5 Static Model Pool

The static track uses a fixed pool of eleven concrete models grouped into four capability tiers (Table A4). Target tier labels are defined existentially over this pool: during the downgrade-and-cascade procedure of §4, we accept a row as belonging to tier $t$ as soon as any one model in that tier's pool correctly handles the

**Algorithm A1** Sequential-locking downgrade search (per trajectory)

**Require:** trajectory $\tau$, hints $h_1, \ldots, h_N$, harness $\mathcal{H}$, tiers $T_0 < T_1 < T_2 < T_3$
locked$[i] \leftarrow T_3$ for all $i$
**for** $i = 1$ to $N$ **do**
  **if** $h_i$ = not-downgradeable **then**
    **continue**
  **end if**
  **for** $t$ from $h_i$ **upto** $T_2$ **do**
    run $\mathcal{H}$ with steps $< i$ locked, step $i$ at $t$, steps $> i$ at $T_3$
    **if** trajectory passes **then**
      locked$[i] \leftarrow t$; **break**
    **end if**
  **end for**
**end for**
**return** locked

current step, as inferred from the full trajectory still passing with the same number of steps. Public static rows store only the resulting tier label rather than any specific model ID, so routers trained on this data remain portable when the underlying pool is updated.

**Table A4.** Static model pool (eleven models, four tiers). Accepted aliases for the same logical model are shown in parentheses. Dynamic-track pricing for representative models is given separately in Table A6.

| Tier | Models |
|---|---|
| `high` | `anthropic/claude-opus-4.6` (alias: `anthropic/claude-opus-4-6`); `openai/gpt-5.4` (alias: `openai/gpt-5.4-2026-03-05`) |
| `mid_high` | `anthropic/claude-haiku-4-5`; `google/gemini-3-flash-preview` (alias: `gemini/gemini-3-flash-preview`); `qwen/qwen3.5-397b-a17b` |
| `mid` | `minimax/minimax-m2.5`; `qwen/qwen3.5-27b`; `qwen/qwen3-coder` |
| `low` | `deepseek/deepseek-v3.2`; `z-ai/glm-4.5-air`; `qwen/qwen3.5-9b` |

## A.6 Static tier accounting rates

| tier | input ($/1M) | cache_read ($/1M) | cache_write ($/1M) | output ($/1M) |
|---|---|---|---|---|
| low | 0.26 | 0.13 | 0.26 | 0.5 |
| mid | 0.30 | 0.059 | 0.30 | 2.0 |
| mid_high | 0.50 | 0.05 | 0.083 | 5.0 |
| high | 5.0 | 0.50 | 6.25 | 25.0 |

**Table A5.** Per-tier accounting constants (USD per million tokens) for the static scoring protocol. These rates are not the prices of any single model; they are representative values derived from the pricing ranges of multiple models in each tier. The `high` tier is 10–50× more expensive than the lower three, which are within 2–5× of each other. Dynamic-pool model prices are in Table A6. This asymmetry drives the ablation in Appendix A.15.

## A.7 Dynamic Pool Representative Models and Pricing

## A.8 Rule-based UncommonRoute Configuration

Rule-based UncommonRoute is run with `MetadataSignal` and `EmbeddingSignal` in a two-signal ensemble, using the upstream defaults `risk_tolerance`$= 0.5$, `ensemble_weights`$= (0.55, 0.45)$, and `low_escalation_threshold`$= 0.55$. Because the benchmark-specific seed set, classifier, and Platt scaler are absent in this rule-based setting, the embedding signal abstains and only the metadata signal contributes in practice.

**Table A6.** Concrete representative models and live OpenRouter unit prices for the TwinRouterBench dynamic pool. These model prices are distinct from the static tier-accounting constants in Table A5. Prices are USD per million tokens; cache-write prices fall back to input prices where the provider does not publish a separate cache-write rate. These are the concrete $p$ values substituted into $c_i(t)$ when scoring the dynamic track; the released scorer ranks the resulting leaderboard bill including the flat unresolved penalty cost (§5.2). The `mid` representative `minimax-m2.7` was selected because it ranks highest among models at a comparable price point on the SWE-bench leaderboard and falls within the same tier as `minimax-m2.5` (the mid-tier representative used during static label construction); the static pool in Table A4 is frozen at the version used when the tier labels were constructed.

| Tier | Representative model | Context | Max out | Input | Output | Cache read | Cache write |
|---|---|---|---|---|---|---|---|
| high | anthropic/claude-opus-4.6 | 1M | 128K | 5.00 | 25.00 | 0.50 | 6.25 |
| mid_high | google/gemini-3-flash-preview | 1.05M | 65.5K | 0.50 | 3.00 | 0.05 | 0.0833 |
| mid | minimax/minimax-m2.7 | 196.6K | 196.6K | 0.30 | 1.20 | 0.059 | 0.30 |
| low | deepseek/deepseek-v3.2 | 163.8K | 163.8K | 0.252 | 0.378 | 0.0252 | 0.252 |

## A.9 Step-Level Cost Case Study: `sympy__sympy-12096`

Table A7 shows a 13-step SWE-bench trajectory. Plan A is all-high execution with prompt caching; Plan B is verified step-level routing (the tier assignments in the "Tier" column). The step-level routing achieves a 39.6% cost reduction relative to all-high while resolving the issue. Last-step output tokens are estimated ($*$).

**Table A7.** Step-level cost case study for `sympy__sympy-12096`. Plan A = all-`high` with prompt caching. Plan B = verified step-level routing. Token counts are in thousands; costs in USD.

| Step | Tier | Plan A in | Plan A out | Plan A cost | Plan B in | Plan B out | Plan B cost |
|---|---|---|---|---|---|---|---|
| 1 | mid | 1,321 | 112 | $0.0111 | 1,284 | 108 | $0.0005 |
| 2 | mid | 1,744 | 71 | $0.0051 | 1,699 | 68 | $0.0003 |
| 3 | mid_high | 1,846 | 69 | $0.0032 | 1,647 | 66 | $0.0003 |
| 4 | mid_high | 2,502 | 67 | $0.0067 | 2,343 | 65 | $0.0003 |
| 5 | low | 3,287 | 89 | $0.0084 | 3,729 | 87 | $0.0010 |
| 6 | mid_high | 4,103 | 256 | $0.0131 | 3,900 | 254 | $0.0010 |
| 7 | low | 4,512 | 55 | $0.0060 | 5,215 | 55 | $0.0009 |
| 8 | mid_high | 4,612 | 168 | $0.0071 | 4,403 | 168 | $0.0007 |
| 9 | high | 4,921 | 241 | $0.0103 | 4,921 | 241 | $0.0368 |
| 10 | high | 5,149 | 85 | $0.0060 | 5,149 | 85 | $0.0060 |
| 11 | high | 5,591 | 63 | $0.0069 | 5,591 | 63 | $0.0069 |
| 12 | low | 5,653 | 56 | $0.0046 | 6,789 | 59 | $0.0018 |
| 13 | low | 5,864 | 111* | $0.0069 | 7,077 | 109* | $0.0010 |
| Total | – | – | – | **$0.0953** | – | – | **$0.0576** |

## A.10 mtRAG/QMSum Judge Hardening Details

Table A8 documents the changes from the legacy judge rubric to the hardened version used in the final dataset. Each row identifies a specific false-pass failure mode in the legacy judge and the corresponding hardened rule applied in construction.

## A.11 Frontier LLM-as-Router Diagnostic Summary

Table A9 shows the Opus 4.6 predictions on verified-`high` SWE-bench steps; Table A10 provides additional diagnostics. These numbers support the claim that a strong model's unexecuted routing judgment is not a reliable substitute for execution-verified step-level supervision (§6.1).

**Table A8.** mtRAG/QMSum judge hardening. The changes convert weak reference-similarity judging into a conservative task-success predicate aligned with the current-turn/query requirement and RadBench-style faithfulness criteria [Kuo et al., 2025].

| Risk in legacy judge | Hardened rule | Purpose |
|---|---|---|
| No primary criterion | First decide whether the answer completely and correctly resolves the current turn/query | Prevents answer/reference surface resemblance from substituting for task success |
| Reference treated as answer string | Treat reference as coverage hints; evidence/transcript is authoritative | Allows valid paraphrases and rejects reference-following errors |
| Unsupported facts | Hard fail for unsupported crucial numbers, dates, amounts, ratios, votes, or claims | Suppresses hallucinated but plausible-looking answers |
| Evidence contradiction | Hard fail when candidate contradicts retrieved passages or meeting transcript | Enforces faithfulness |
| Unanswerable/no-context cases | mtRAG pre-filter: must be answerable, have context passages, and positive human relevance feedback | Removes structurally unreliable samples before routing search |
| All tiers fail | Mark discarded; do not emit case JSON or supervision row | Prevents false passes from becoming verified labels |

| Verified tier | $\tilde{t} =$ `low` | $\tilde{t} =$ `mid` | $\tilde{t} =$ `mid_high` | $\tilde{t} =$ `high` | Total |
|---|---|---|---|---|---|
| $\hat{t} =$ `high` | 34 | 43 | 63 | **7** | 147 |

**Table A9.** Opus 4.6 predictions on verified-`high` SWE-bench steps. 140 of 147 are routed below `high`.

## A.12 Label-Validation Audit Trail

Table A11 documents the validation mechanisms applied during dataset construction. Together they ensure that no label enters the released JSONL unless the strong baseline passed, a tier-pool model actually passes under the mixed-model trace, the judge does not mark the answer as pseudo-passing, and the final human audit sees no surviving issue.

## A.13 Manual Audit Protocol and Summary

The manual audit is the final quality gate of the static track: it runs *after* the Opus-initialized downgrade search, tier-pool cascade check, open-ended judge hardening, and prefix reconstruction of §4 are all complete. The audit is deliberately a human-judgment pass rather than a second harness execution. Its purpose is to cross-check that the released tier label is still defensible on the same router-visible context that a deployed router would see, not to recompute any pass predicate.

**Sampling.** We follow the GT tier-optimality review protocol included with the release artifacts. The audit unit is a single routed step so that single-step and multi-step cases are treated on the same granularity. Within each of the three workloads that contain multi-step trajectories (BFCL, SWE-bench, PinchBench) we draw an independent random sample of roughly 10% of the workload's routed steps. mtRAG and QMSum are single-turn tasks that lack multi-turn compounding complexity; their labels are already constrained by the hardened judge whose pseudo-pass patterns were eliminated through iterative calibration (§4), and are not resampled in this audit.

**Table A10.** Frontier LLM-as-router (Opus 4.6) diagnostic summary on SWE-bench. The key finding is severe under-prediction of `high`-tier steps: only 5% of verified-`high` steps are predicted `high`, causing every SWE-bench trajectory to fail under this routing.

| Diagnostic | Value | Scope | Interpretation |
|---|---|---|---|
| SWE-bench valid rows | 300 | 336 rows total, 36 malformed | Valid subset for confusion-matrix analysis |
| Verified-high predicted high | 7/147 (4.8%) | SWE-bench valid rows | Severe under-prediction of high-tier steps |
| Verified-high under-predicted | 140/147 (95.2%) | SWE-bench valid rows | Most high steps routed below high |
| SWE-bench trajectory pass | 0/40 | Legacy Opus routing run | Every trajectory has $\geq$1 under-routed step |
| Middle-third under-prediction | 73% | Step-position diagnostic | Core editing/test-running steps most under-predicted |
| First-third under-prediction | 46% | Step-position diagnostic | Lower but still substantial under-routing |
| Last-third under-prediction | 53% | Step-position diagnostic | Recovery/finalization steps remain difficult |

**Review procedure.** For each sampled step, the router-visible `messages` prefix is held fixed at its released form. An optional large-model pre-label may be attached as a reference, but the final decision is always taken by a human reviewer, who asks whether lowering the verified tier by one more level would still produce a response that correctly handles the current step; the step is then labelled as *tight*, *uncertain*, or *further-downgradeable*. A *tight* verdict means the released tier is already at the conditional optimum and cannot be lowered one more step without breaking task resolution; a *further-downgradeable* verdict means the reviewer believes the released tier is still too high and the label can be tightened by one more tier. The audit inspected 64 steps in total; one SWE-bench step was flagged as further-downgradeable and its verified tier was lowered by one tier before release, while the remaining 63 steps were judged tight. This internal audit is intended to catch residual labels that the upstream search did not tighten to the conditional optimum, and to clarify label semantics; it is not presented as an external annotation study or as a confidence interval for the full corpus.

## A.14 Static-Track Breakdown

**Per-workload cost savings.** Table A13 shows that SWE-bench is the hardest static slice. It carries 34.64% of the row-weighted CostSave score, and every router has negative cost savings there because one under-routed step fails the whole trajectory and triggers the failure-aware bill. On the other four workloads, SR-KNN and ClawRouter (rule-based) both exceed 85% cost savings on average; the global ranking depends on whether a router preserves code-repair trajectories.

**Router failure modes.** The three baselines fail in different ways (Table A14). On the 800 rows whose verified tier is below `high`, SR-KNN is mostly exact; ClawRouter (rule-based) almost always routes one tier above the label; UncommonRoute (rule-based) is exact more often than ClawRouter (rule-based) but jumps to `high` 217 times.

SWE-bench exposes why row-level behavior is insufficient (Table A15). UncommonRoute (rule-based) recognizes many verified-`high` steps, but missing even one step in an 8–13 call trajectory fails the instance.

**Table A11.** Label-validation audit trail. Each component addresses a specific failure mode in the construction pipeline.

| Validation component | Scope | Failure mode addressed | Release action |
|---|---|---|---|
| Successful seed traces | All released instances | Conflating task failure with routing failure | Only passing seed traces enter degradation search |
| Sequential-locking downgrade search | Multi-step traces | Missing the last-passing tier; hypothetical overlays on the strongest trace | Push one tier below the Opus proposal under the causal mixed-model prefix until the harness fails, then lock the last passing tier |
| Tier cascade (3 models/tier) | All non-`low` tier labels | Single-model false negatives within a tier | Lock a cheaper tier only after an actual passing run |
| Strict mtRAG/QMSum judge | Open-ended RAG and summarization | False passes from weak judging | Use hard-fail rubric; discard all-tier failures |
| Final human audit | ∼10% random step samples from each of BFCL, SWE-bench, and PinchBench | Still-downgradeable tier labels that the upstream search did not tighten to the conditional optimum | Human review under the router-visible prefix decides whether the released tier can be lowered by one more step; one SWE-bench step was flagged as further-downgradeable and its tier was lowered accordingly |

## A.15 Cost-Savings Accounting Ablation

This section provides the full derivation and per-variant numbers referenced in §6.1.

**Pricing asymmetry drives the ablation.** Table A16 reports, for three representative step profiles, the per-step saving against the always-`high` baseline under each target tier. The ratio `save_mid / save_low` is within 3% of 1 across all three step sizes, which means a router that mis-routes from $\hat{t} = \texttt{low}$ to $\tilde{t} = \texttt{mid}$ loses almost no money in absolute terms.

**Five scoring variants.** We compare five $N/D$ accounting schemes on the same predictions:

- **A** (legacy row-level denominator): $D = \sum_i (\text{cost}(3; i) - \text{cost}(\hat{t}_i; i))$ over passing rows with $\hat{t}_i \neq \texttt{high}$; $N$ ignores failures.
- **B** (legacy all-row denominator): $D$ as in A but over all rows; failing trajectories subtract $\sum_i \text{cost}(\tilde{t}_i; i)$ at trajectory level.
- **C** (intermediate high-row exclusion): exclude $\hat{t}_i = \texttt{high}$ rows from $D$.
- **D** ($D = \sum_i \text{cost}(3; i)$, failed-step penalty at step level, no trajectory-level penalty).

**Table A12.** Manual audit summary. A sampled step is counted as *Downgradeable* when the reviewer believes its verified tier can still be lowered by one more tier without breaking task resolution; *Tight %* is the complementary share whose tier label is already at the conditional optimum. Sampling is done at the step granularity across the three workloads that contain multi-step trajectories; mtRAG and QMSum are single-turn tasks already guarded by the hardened judge and are not resampled here. The audit is a targeted internal quality check.

| Slice | Total steps | Sampled steps | Sampling % | Downgradeable | Tight % |
|---|---|---|---|---|---|
| BFCL | 248 | 25 | 10.1 | 0 | 100.0 |
| SWE-bench | 336 | 34 | 10.1 | 1 | 97.1 |
| PinchBench | 48 | 5 | 10.4 | 0 | 100.0 |
| Total | 632 | 64 | 10.1 | 1 | 98.4 |

| Router | BFCL | mtRAG | QMSum | Pinch | SWE-bench | Overall |
|---|---|---|---|---|---|---|
| SR-KNN | **90.49** | **92.35** | 90.24 | 35.36 | **-1.64** | **56.18** |
| ClawRouter (rule-based) | 89.87 | 83.45 | **92.12** | **55.20** | -6.55 | 53.82 |
| UncommonRoute (rule-based) | 47.14 | 91.93 | 90.24 | 39.80 | -86.08 | 15.99 |
| Row weight | 25.57 | 19.90 | 14.95 | 4.95 | 34.64 | 100.00 |

**Table A13.** Per-workload CostSave under the static scoring protocol. The SWE-bench slice is the main stress test: failed trajectories trigger the failure-aware numerator in §5.1.

- **E** (final failure-aware formula): same $D$ as D, but failed trajectories charge $-\sum_i \text{cost}(\tilde{t}_i; i)$ at trajectory level.

**Why variant E recovers the consistent ranking.** Variant E interprets a failing trajectory as "the router burned its own (cheap) budget *and* the user paid the full `high` bill to recompute." Formally, the user-side bill on a failing trajectory is $\sum_i \text{cost}(\tilde{t}_i; i) + \sum_i \text{cost}(3; i)$, so savings against always-`high` equal $-\sum_i \text{cost}(\tilde{t}_i; i)$, which is exactly what $N$ encodes. The $\sum_i \text{cost}(3; i)$ retry term is absorbed by $D$ rather than double-counted, keeping CostSave $\in [-\infty, 100]$ with 0 representing always-`high` routing. On the full benchmark, ClawRouter (rule-based) has 649 up-one errors (near-free in absolute terms) that can no longer compensate for its 38 failing SWE-bench trajectories, and its CostSave aligns with its RowPass/RowExact/TrajPass figures.

## A.16 Opus 4.6 Confusion Matrix

For reference, the Opus 4.6 routing run recorded under an earlier three-metric accounting protocol reports an overall of 84.05, with row pass =85.77, row exact =82.88, and cost savings =83.48. This is not directly comparable to the static scores in Table 2 because the earlier accounting does not apply the failure-aware trajectory penalty in §5.1 and does not report a trajectory-pass metric; we therefore treat it as a historical data point rather than a ranked baseline.

Table A18 shows the full confusion matrix summarized in §6.1.

## A.17 mtRAG Row Example

A second row format, complementing the SWE-bench example in §3, illustrates how `low`-tier labels arise: after several retrieval turns, a short follow-up query can be answered by the cheapest tier in the pool.

| Prediction pattern on $\hat{t} <$ `high` | SR-KNN | ClawRouter (rule-based) | UncommonRoute (rule-based) |
|---|---|---|---|
| Exact tier | **627** | 107 | 488 |
| Up one tier | 10 | **649** | 47 |
| Jump to `high` | 117 | 21 | **217** |
| Fail: predicted below $\hat{t}$ | 46 | 23 | **48** |

**Table A14.** Error patterns on the 800 non-`high` rows.

| Router | Predicted `high` on $\hat{t} =$ `high` | Hit rate | Failed traj. |
|---|---|---|---|
| SR-KNN | 137/168 | **81.5%** | **10/40** |
| ClawRouter (rule-based) | 7/168 | 4.2% | 38/40 |
| UncommonRoute (rule-based) | 105/168 | 62.5% | 39/40 |

**Table A15.** SWE-bench high-tier decisions. One under-routed critical step fails the whole trajectory.


```
{
  "id": "mtrag_..._turn_3_step_1",
  "benchmark": "mtrag",
  "step_index": 1, "total_steps": 1,
  "messages": [
    {"role": "system", "content": "Answer based on the retrieved passages..."},
    {"role": "user",   "content": "[Passage] Major.minor update... [Passage] ..."},
    {"role": "assistant", "content": "..."},
    ...multi-turn history...
    {"role": "user",   "content": "Major.minor update."}
  ],
  "target_tier": "low", "target_tier_id": 0
}
```

**Table A16.** Savings vs. always-`high` for different target tiers on three representative step profiles. Mis-routing from `low` to `mid` is nearly cost-free.

| Step profile | save(high−low) | save(high−mid) | save(high−mid_high) | save_mid / save_low |
|---|---|---|---|---|
| 4k cold step | 3.621¢ | 3.530¢ | 3.467¢ | 0.975 |
| 20k warm step | 1.965¢ | 2.032¢ | 1.900¢ | 1.034 |
| 100k cold step | 61.13¢ | 60.65¢ | 62.67¢ | 0.992 |

**Table A17.** Ablation of COSTSAVE accounting. Variants A–D all rank ClawRouter (rule-based) above SR-KNN; only variant E (the final failure-aware formula) aligns with ROWPASS, ROWEXACT, and TRAJPASS.

| Variant | SR-KNN COST% | ClawRouter (rule-based) COST% | Ranking |
|---|---|---|---|
| A (legacy row-level denominator) | 84.99 | **89.71** | Claw wins |
| B (legacy all-row denominator) | 84.45 | **84.59** | Claw wins |
| C (intermediate high-row exclusion) | 79.11 | **91.31** | Claw wins |
| D ($D = \sum \text{cost}(3)$, step-level fail) | 61.81 | **67.67** | Claw wins |
| **E** (final failure-aware formula) | **56.18** | 53.82 | SR wins |

**Table A18.** Opus 4.6 routing predictions vs. verified tiers on the 300 valid SWE-bench rows (36 malformed outputs excluded). Diagonal cells are highlighted. Opus rarely commits to `high`, even when the verified tier is `high`.

| | $\tilde{t} =$ `low` | $\tilde{t} =$ `mid` | $\tilde{t} =$ `mid_high` | $\tilde{t} =$ `high` | row total |
|---|---|---|---|---|---|
| $\hat{t} =$ `low` | **52 (61%)** | 15 | 17 | 1 | 85 |
| $\hat{t} =$ `mid` | 13 | **7 (23%)** | 10 | 0 | 30 |
| $\hat{t} =$ `mid_high` | 8 | 13 | **16 (42%)** | 1 | 38 |
| $\hat{t} =$ `high` | 34 | 43 | 63 | **7 (5%)** | 147 |